

Top 50 Interview Questions

on **Agentic AI**

This single document is all you need to
crack your Agentic AI interviews



AI Agents



RAG



LangChain



LangGraph



CrewAI



Autogen



Naved Khan
Gen AI Engineer

+ Follow



Section 1

Agentic AI Fundamentals

Questions 1-10

What is Agentic AI and how does it differ from traditional AI?

Agentic AI refers to AI systems that operate **autonomously** to achieve goals without continuous human guidance. They perceive environment, make decisions, take actions, and learn from outcomes.

KEY DIFFERENCE

Traditional AI (like ChatGPT) responds reactively to individual prompts. Agentic AI maintains state, creates multi-step plans, uses external tools, and adapts based on results.

- ✓ **Traditional AI:** "Book a flight" → provides booking info
- ✓ **Agentic AI:** Searches flights, compares prices, books, sends confirmation



Naved Khan
Gen AI Engineer

1/50

+ Follow

What are the core components of an AI agent?

An AI agent consists of **five essential components** working together:

- ✓ **Perception Module:** Data gathering from environment (user inputs, API responses, documents)
- ✓ **Reasoning & Planning:** Brain of agent - analyzes state, evaluates actions, creates step-by-step plans
- ✓ **Memory System:** Short-term (conversation) + Long-term (vector stores)
- ✓ **Action Module:** Executes decisions - APIs, code, messages
- ✓ **Learning Module:** Improves performance through feedback



Naved Khan
Gen AI Engineer

2/50

+ Follow

Explain the ReAct (Reasoning + Acting) pattern

ReAct interleaves reasoning steps with action execution. The agent alternates between thinking and doing.

THE LOOP

Thought → Action → Observation → Thought → ...

```
# Example: "Population of Tokyo × 2?"
```

PYTHON

Thought: Need Tokyo's population first

Action: Search["Tokyo population"]

Observation: ~14 million

Thought: Now multiply by 2

Action: Calculator["14000000 * 2"]

Observation: 28000000

Thought: Have final answer

Action: Finish["28 million"]



What are the different types of AI agents?

- ✓ **Simple Reflex:** Condition-action rules, no memory (e.g., thermostat)
- ✓ **Model-Based Reflex:** Maintains internal world model (e.g., robot vacuum)
- ✓ **Goal-Based:** Plans actions to achieve explicit goals (e.g., navigation)
- ✓ **Utility-Based:** Optimizes utility function, handles trade-offs
- ✓ **Learning Agents:** Improves through experience (e.g., game AI)
- ✓ **Multi-Agent Systems:** Multiple agents collaborating/competing



Naved Khan
Gen AI Engineer

4/50

+ Follow

How does tool use and function calling work in LLM-based agents?

Tool use extends LLM capabilities by allowing interaction with external systems.

```
PYTHON
# 1. Define tool schema
tool_schema = {
    "name": "get_weather",
    "description": "Get weather for location",
    "parameters": {"location": "string"}
}

# 2. LLM decides to call tool
# User: "What's the weather in Paris?"
llm_output = {"tool": "get_weather",
              "args": {"location": "Paris"}}

# 3. Execute & return result
result = "15°C, partly cloudy"

# 4. LLM generates final response
```

5/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

What is memory management in AI agents?

Memory enables agents to maintain context and learn from interactions.

- ✓ **Short-Term:** Current conversation context (within token limits)
- ✓ **Long-Term:** Persists across sessions using vector databases
- ✓ **Episodic:** Records specific experiences and outcomes
- ✓ **Semantic:** General facts and world knowledge

MEMORY STRATEGIES

Sliding window • Summarization • Selective persistence • Hybrid approaches



How do agents handle error recovery and fault tolerance?

- ✓ **Retry with Backoff:** Wait and retry with exponentially increasing delays (1s, 2s, 4s...)
- ✓ **Fallback Mechanisms:** Alternative paths when primary approaches fail
- ✓ **Checkpointing:** Save state at critical points to resume from
- ✓ **Output Validation:** Verify tool results and LLM outputs are sensible
- ✓ **Graceful Degradation:** Provide partial results or helpful error messages
- ✓ **Human-in-the-Loop:** Escalate to humans when automated recovery fails

7/50



Naved Khan
Gen AI Engineer

+ Follow

Explain planning strategies used in AI agents

- ✓ **Chain-of-Thought (CoT):** Step-by-step reasoning before answering. Linear thinking.
- ✓ **Tree-of-Thoughts (ToT):** Explores multiple reasoning paths, can backtrack.
- ✓ **ReAct:** Planning interleaved with execution, adapts based on feedback.
- ✓ **Plan-and-Execute:** Create complete plan first, then execute step by step.
- ✓ **Hierarchical:** Decompose goals into sub-goals recursively.

CHOOSING STRATEGY

Simple tasks → CoT | Uncertain environments → ReAct | Complex projects → Hierarchical



How is AI agent performance evaluated?

Task Success Rate: % of tasks completed correctly (primary metric)

- ✓ **Efficiency:** Steps taken, token usage, time, API calls
- ✓ **Quality:** Accuracy, relevance, completeness
- ✓ **Safety:** Error rate, hallucination rate, policy violations
- ✓ **User Experience:** Satisfaction, abandonment rate, human intervention need

EVALUATION TIP

Use RAGAS for automated evaluation. Test with diverse cases: normal ops, edge cases, adversarial inputs.



What are security best practices for deploying Agentic AI?

- ✓ **Prompt Injection Defense:** Input validation, instruction separation
- ✓ **Least Privilege:** Only necessary capabilities per agent
- ✓ **Output Filtering:** Scan for PII, API keys, harmful content
- ✓ **Sandboxed Execution:** Run code in Docker/isolated environments
- ✓ **Rate Limiting:** Prevent abuse, control costs
- ✓ **Audit Logging:** Record all actions for accountability
- ✓ **Human Oversight:** Approval for sensitive actions

10/50



Naved Khan
Gen AI Engineer

+ Follow



Section 2

RAG - Retrieval Augmented Generation

Questions 11-20

What is RAG and why is it important?

RAG enhances LLM responses by incorporating information retrieved from external knowledge sources at query time.

THREE STAGES

1. RETRIEVAL → 2. AUGMENTATION → 3. GENERATION

- ✓ **Hallucination Reduction:** Grounds responses in retrieved facts
- ✓ **Knowledge Currency:** Access current info without retraining
- ✓ **Domain Specialization:** Index internal documents
- ✓ **Cost Effective:** Cheaper than fine-tuning



What are the different RAG architectures?

- ✓ **Naive RAG:** Simple retrieve → stuff prompt → generate
- ✓ **Advanced RAG:** Query rewriting + Re-ranking
- ✓ **Modular RAG:** Swappable components
- ✓ **Iterative RAG:** Multiple retrieval rounds
- ✓ **Fusion RAG:** Multiple query variations + fuse results
- ✓ **Corrective RAG (CRAG):** Evaluates & corrects retrieval
- ✓ **Agentic RAG:** Agent controls entire RAG process



How should documents be chunked for RAG?

- ✓ **Fixed-Size:** Split by character/token count (simple)
- ✓ **Recursive:** Try separators in sequence (most common)
- ✓ **Semantic:** Use embeddings for topic boundaries
- ✓ **Structural:** Respect document structure (headers, sections)

KEY PARAMETERS

Chunk Size: 500-1000 tokens typical

Overlap: 10-20% to preserve context

Metadata: Store source, page, section



What is hybrid search in RAG?

Hybrid search combines dense (vector) + sparse (keyword) retrieval:

- ✓ **Dense:** Semantic similarity via embeddings (handles synonyms)
- ✓ **Sparse (BM25):** Keyword matching (exact terms, codes)

```
# Weighted combination
final_score = a × dense_score + (1-a) × sparse_score

# Reciprocal Rank Fusion (RRF)
score =  $\sum 1/(k + \text{rank}_i)$ 
```

PYTHON

WHY HYBRID?

Consistently outperforms either method alone – queries vary from conceptual to specific.



How is RAG system performance evaluated?

Retrieval Metrics:

- ✓ **Precision:** % of retrieved docs that are relevant
- ✓ **Recall:** % of relevant docs that were retrieved
- ✓ **MRR:** Where does first relevant doc appear?

Generation Metrics:

- ✓ **Faithfulness:** Answer supported by retrieved context?
- ✓ **Answer/Context Relevance:** On topic?

USE RAGAS

Automated evaluation framework combining these metrics



What vector databases are used in RAG?

- ✓ **Pinecone:** Managed cloud, auto-scaling, minimal ops
- ✓ **Chroma:** Lightweight, embedded, great for prototyping
- ✓ **Weaviate:** Open-source, built-in vectorization, GraphQL
- ✓ **Qdrant:** High-performance Rust, strong filtering
- ✓ **Milvus:** Enterprise-grade, billions of vectors, GPU

SELECTION FACTORS

Scale • Self-hosted vs Managed • Hybrid search • Filtering • Cost • Latency



How are embedding models chosen for RAG?

- ✓ **OpenAI Ada-002:** 1536-dim, strong general, paid API
- ✓ **BGE Models:** State-of-the-art open-source, various sizes
- ✓ **Sentence-Transformers:** Many pre-trained, can fine-tune
- ✓ **Cohere Embed:** Multilingual, task-specific variants

SELECTION CRITERIA

Quality • Dimensions (384-1536) • Domain fit • Latency • Cost •
Language support



Naved Khan
Gen AI Engineer

17/50

+ Follow

How are RAG systems optimized for production?

- ✓ **Query Optimization:** HyDE, query expansion, reformulation
- ✓ **Two-Stage Retrieval:** Fast retrieval → sophisticated re-ranking
- ✓ **Context Compression:** Extract only relevant sentences
- ✓ **Caching:** Cache responses & embeddings for frequent queries
- ✓ **Streaming:** Return partial responses as they generate
- ✓ **Async Processing:** Concurrent operations
- ✓ **Index Optimization:** HNSW, IVF tuning for dataset size



What are common challenges in RAG systems?

- ✓ **Chunking Issues:** Context spans boundaries → use overlap, semantic chunking
- ✓ **Retrieval Failures:** Different terminology → hybrid search, query expansion
- ✓ **Context Window Limits:** Important info excluded → compression, re-ranking
- ✓ **Lost in the Middle:** LLMs ignore middle content → place key info at start/end
- ✓ **Hallucination Despite Context:** LLM fills gaps → grounding instructions
- ✓ **Stale Information:** Outdated index → regular reindexing



Explain advanced RAG techniques

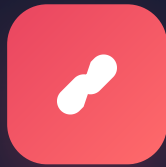
- ✓ **Self-RAG:** Self-reflection, evaluates quality, regenerates if needed
- ✓ **RAPTOR:** Hierarchical tree of summaries, multi-level retrieval
- ✓ **Graph RAG:** Knowledge graph for relationships, graph traversal
- ✓ **Adaptive RAG:** Routes queries to appropriate strategies
- ✓ **Corrective RAG:** Evaluates retrieval, triggers fallbacks
- ✓ **Agentic RAG:** Agent controls when/what/how to retrieve

20/50



Naved Khan
Gen AI Engineer

+ Follow



Section 3

LangChain Framework

Questions 21-28

What is LangChain and what problems does it solve?

LangChain is an open-source framework for building LLM-powered applications.

- ✓ **Integration:** Unified interfaces for LLM providers, vector stores, APIs
- ✓ **Prompts:** Templates with variables, few-shot examples
- ✓ **Chains:** Sequences of operations (LCEL)
- ✓ **Agents:** Autonomous entities using tools
- ✓ **Memory:** Conversation context management



What is LCEL (LangChain Expression Language)?

LCEL is a declarative way to compose chains using the **pipe operator (|)**

```
# Traditional approach
chain = LLMChain(llm=llm, prompt=prompt,
                 output_parser=parser)

# LCEL approach - cleaner!
chain = prompt | llm | parser

# Complex composition
chain = {
    "context": retriever,
    "question": RunnablePassthrough()
} | prompt | llm | parser
```

PYTHON

LCEL BENEFITS

Streaming • Async support • Parallel execution • Built-in retries &

22/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

How does memory work in LangChain?

- ✓ **ConversationBufferMemory:** Complete history (unbounded)
- ✓ **BufferWindowMemory:** Last K interactions only
- ✓ **SummaryMemory:** LLM summarizes older parts
- ✓ **EntityMemory:** Tracks entities (people, places)
- ✓ **VectorStoreMemory:** Retrieves relevant past messages

```
memory = ConversationBufferWindowMemory(k=5)
chain = ConversationChain(llm=llm, memory=memory)
chain.predict(input="My name is Alex")
chain.predict(input="What's my name?") # Alex!
```

PYTHON



Naved Khan
Gen AI Engineer

23/50

[+ Follow](#)

How are agents created in LangChain?

```
tools = [  
    Tool(name="Search", func=search_api.run,  
          description="Search for current info"),  
    Tool(name="Calculator", func=calc.run,  
          description="Math calculations")  
]  
  
agent = initialize_agent(  
    tools=tools,  
    llm=llm,  
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,  
    verbose=True  
)  
  
result = agent.run("Population of France / 1000?")
```

AGENT TYPES

ZERO_SHOT_REACT • CONVERSATIONAL •
OPENAI_FUNCTIONS • STRUCTURED_CHAT

24/50



Naved Khan
Gen AI Engineer

+ Follow

How are custom tools created in LangChain?

PYTHON

```
from langchain.tools import tool

# Simple decorator approach
@tool
def multiply(a: int, b: int) → int:
    """Multiply two numbers together."""
    return a * b

# Structured tool with Pydantic
class SearchInput(BaseModel):
    query: str
    max_results: int = 5

search_tool = StructuredTool.from_function(
    func=search,
    args_schema=SearchInput,
    description="Search for information"
)
```

25/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

What are callbacks in LangChain?

Callbacks are hooks that execute during chain/agent execution for logging, monitoring, streaming.

PYTHON

```
class CostTracker(BaseCallbackHandler):  
    def __init__(self):  
        self.total_tokens = 0  
  
    def on_llm_end(self, response, **kwargs):  
        tokens = response.llm_output.get(  
            "token_usage", {})  
        self.total_tokens += tokens.get(  
            "total_tokens", 0)  
  
tracker = CostTracker()  
chain.invoke(input, config={"callbacks": [tracker]})
```

USE CASES

Streaming • Logging • Cost tracking • Debugging • LangSmith tracing

26/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

How are prompts managed in LangChain?

PYTHON

```
# PromptTemplate for completion models
prompt = PromptTemplate(
    template="You are expert on {topic}.\n{question}",
    input_variables=["topic", "question"]
)

# ChatPromptTemplate for chat models
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a {domain} expert."),
    ("human", "{question}")
])

# FewShotPromptTemplate with examples
few_shot = FewShotPromptTemplate(
    examples=[{"in": "happy", "out": "sad"}],
    ...
)
```

BEST PRACTICE

27/50



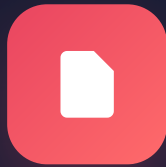
Naved Khan
Gen AI Engineer

[+ Follow](#)

What are best practices for LangChain in production?

- ✓ **Use LCEL:** Modern syntax with built-in streaming & async
- ✓ **Error Handling:** `.with_retry()` and `.with_fallback()`
- ✓ **Observability:** LangSmith for tracing, monitoring
- ✓ **Caching:** Cache LLM responses for repeated queries
- ✓ **Token Optimization:** Shorter prompts, compress context
- ✓ **Output Validation:** Verify responses before using
- ✓ **Rate Limiting:** Prevent abuse and control costs





Section 4

LangGraph Framework

Questions 29-36

What is LangGraph and how does it differ from LangChain?

LangGraph is for building stateful, multi-agent applications with complex control flow.

- ✓ **LangChain:** Sequential chains (DAGs), linear flow
- ✓ **LangGraph:** Supports cycles, loops back to previous steps

GRAPH STRUCTURE

NODES: Functions that process state

EDGES: Define transitions between nodes

STATE: Flows through graph, accumulates results



How does state management work in LangGraph?

PYTHON

```
from typing import TypedDict, Annotated
import operator

class AgentState(TypedDict):
    # Annotated with operator.add = append
    messages: Annotated[list, operator.add]
    current_step: str
    results: dict

def process_node(state: AgentState) → AgentState:
    messages = state["messages"]
    # Process...
    return {"messages": [new_message]} # Appended!
```

STATE FLOW

Initial state → Each node receives & updates → State passes to next node

30/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

How is conditional routing implemented in LangGraph?

PYTHON

```
def should_continue(state):  
    """Routing function returns next node name"""  
    if state["complete"]:  
        return "end"  
    elif state["needs_review"]:  
        return "review"  
    else:  
        return "process"  
  
graph.add_conditional_edges(  
    "check",          # Source node  
    should_continue,  # Routing function  
    {  
        "end": END,  
        "review": "review_node",  
        "process": "process_node"  
    }  
)
```

31/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

What is checkpointing in LangGraph?

Checkpointing saves graph execution state for interruption, resumption, and debugging.

```
PYTHON
from langgraph.checkpoint.memory import MemorySaver

memory = MemorySaver()
app = graph.compile(checkpointer=memory)

# Run with thread_id for persistence
config = {"configurable": {"thread_id": "user123"}}
result = app.invoke(initial_state, config)

# Later, resume from checkpoint
result = app.invoke({"continue": True}, config)
```

USE CASES

Long-running workflows • Human-in-the-loop • Time-travel debugging



How is human-in-the-loop implemented in LangGraph?

Pauses execution for human input before continuing.

PYTHON

```
graph.add_node("get_approval", get_approval_node)
graph.add_node("execute", execute_node)

graph.add_conditional_edges(
    "get_approval",
    lambda s: "execute" if s.get("approved") else "reject",
    {"execute": "execute", "reject": "reject_node"}
)

# Application code
result = app.invoke(state, config)
if result.get("waiting_for_approval"):
    approval = get_human_approval() # UI
    app.invoke({"approved": approval}, config)
```

33/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

How are multi-agent systems built in LangGraph?

- ✓ **Supervisor Pattern:** One agent coordinates workers, decides who handles each task
- ✓ **Collaborative:** Agents build on each other (Research → Write → Review)
- ✓ **Debate Pattern:** Agents argue positions to reach better conclusions

```
graph.add_node("researcher", researcher_agent)
graph.add_node("writer", writer_agent)
graph.add_node("coordinator", coordinator)
graph.add_conditional_edges(
    "coordinator", route_to_agent,
    {"researcher": "researcher", ...}
)
```

PYTHON



Naved Khan
Gen AI Engineer

34/50

[+ Follow](#)

What are subgraphs in LangGraph?

components.

PYTHON

```
# Define subgraph
research_graph = StateGraph(ResearchState)
research_graph.add_node("search", search_node)
research_graph.add_node("analyze", analyze_node)
research_graph.add_edge("search", "analyze")
research_subgraph = research_graph.compile()

# Use in parent graph
main_graph = StateGraph(MainState)
main_graph.add_node("research", research_subgraph)
main_graph.add_node("write", write_node)
main_graph.add_edge("research", "write")
```

BENEFITS

Reusability • Isolated testing • Better organization •
Encapsulation

35/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

When should LangGraph be used vs LangChain?

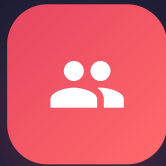
Use LangChain when:

- ✓ Simple sequential workflows ($A \rightarrow B \rightarrow C$)
- ✓ Basic RAG, quick prototypes, standard agents

Use LangGraph when:

- ✓ Workflow has loops (retry until success)
- ✓ Multiple agents collaborate
- ✓ Need pause/resume, checkpointing
- ✓ Complex conditional routing, state persistence





Section 5

CrewAI Framework

Questions 37-43

What is CrewAI?

CrewAI orchestrates multiple AI agents that work together as a "crew" to accomplish complex tasks.

- ✓ **Agents:** Individual AI with roles, goals, and capabilities
- ✓ **Tasks:** Specific assignments with expected outputs
- ✓ **Crews:** Teams of agents working together
- ✓ **Tools:** Search, code execution, file operations
- ✓ **Processes:** Sequential or hierarchical execution

EXCELS AT

Research • Content creation • Code review • Complex problem decomposition

37/50



Naved Khan
Gen AI Engineer

+ Follow

How are agents defined in CrewAI?

```
from crewai import Agent

researcher = Agent(
    role="Senior Research Analyst",
    goal="Discover and analyze AI trends",
    backstory="""Expert researcher with 10 years
experience in AI. Known for thorough analysis
and attention to detail.""",
    tools=[search_tool, web_scraper],
    llm=llm,
    verbose=True
)

writer = Agent(
    role="Technical Writer",
    goal="Create clear, engaging content",
    backstory="Makes complex topics accessible",
    allow_delegation=True
)
```

PYTHON

38/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

How are tasks defined in CrewAI?

```
from crewai import Task

research_task = Task(
    description="""Research top 5 AI trends for 2025.
    Focus on practical business applications.
    Include statistics and expert opinions.""",
    expected_output="""Comprehensive report:
    - 5 key trends with descriptions
    - Supporting data for each""",
    agent=researcher
)

writing_task = Task(
    description="Write blog post from research",
    expected_output="1500-word blog post",
    agent=writer,
    context=[research_task] # Uses research output!
)
```

PYTHON



What are the process types in CrewAI?

✓ **Hierarchical:** Manager agent coordinates workers dynamically

PYTHON

```
from crewai import Crew, Process

# Sequential - fixed order
crew = Crew(
    agents=[researcher, writer, editor],
    tasks=[research_task, writing_task, editing_task],
    process=Process.sequential
)

# Hierarchical - manager decides
crew = Crew(
    agents=[researcher, writer],
    tasks=[tasks],
    process=Process.hierarchical,
    manager_llm=llm
)

result = crew.kickoff()
```

40/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

How do agents collaborate in CrewAI?

- ✓ **Task Context:** Pass output from earlier tasks to later ones
- ✓ **Delegation:** Agents can assign work to teammates
- ✓ **Memory:** Shared memory accessible to all agents

```
writing_task = Task(  
    ...,  
    context=[research_task] # Gets research output  
)  
  
agent = Agent(  
    ...,  
    allow_delegation=True # Can delegate  
)  
  
crew = Crew(..., memory=True) # Shared memory
```

PYTHON



Naved Khan
Gen AI Engineer

41/50

[+ Follow](#)

How are custom tools added to CrewAI?

PYTHON

```
from crewai.tools import tool, BaseTool

# Using @tool decorator
@tool
def search_database(query: str) → str:
    """Search company database for info.
    Args:
        query: Search query string
    Returns:
        Matching records"""
    return database.search(query)

# Class-based tool
class CalculatorTool(BaseTool):
    name: str = "Calculator"
    description: str = "Math calculations"

    def _run(self, expression: str) → str:
        return str(eval(expression))
```

42/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

What are best practices for CrewAI?

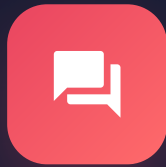
- ✓ **Clear Roles:** Distinct responsibilities, no overlap
- ✓ **Detailed Backstories:** Expertise, personality, approach
- ✓ **Small Tasks:** Break complex into focused subtasks
- ✓ **Hierarchical for Complex:** Manager makes better decisions
- ✓ **Set Constraints:** max_iterations, max_rpm, timeout
- ✓ **Monitor Tokens:** Multi-agent = high consumption
- ✓ **Test Individually:** Verify each agent before combining

43/50



Naved Khan
Gen AI Engineer

+ Follow



Section 6

Autogen Framework

Questions 44-50

What is Autogen?

Autogen is Microsoft's framework for building multi-agent conversational systems. Agents communicate through natural conversation.

- ✓ **ConversableAgent:** Base agent for send/receive messages
- ✓ **AssistantAgent:** AI-powered using LLMs
- ✓ **UserProxyAgent:** Represents humans, executes code
- ✓ **GroupChat:** Multi-agent conversations

EXCELS AT

Pair programming • Collaborative problem solving • Code generation & execution

44/50



Naved Khan
Gen AI Engineer

+ Follow

How are conversable agents created in Autogen?

```
PYTHON
from autogen import AssistantAgent, UserProxyAgent

# AI-powered assistant
assistant = AssistantAgent(
    name="Assistant",
    llm_config={"model": "gpt-4"},
    system_message="You are a helpful AI."
)

# User proxy (can execute code)
user_proxy = UserProxyAgent(
    name="User",
    human_input_mode="NEVER", # Automated
    code_execution_config={
        "work_dir": "coding",
        "use_docker": True
    }
)
```

45/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

How does group chat work in Autogen?

PYTHON

```
from autogen import GroupChat, GroupChatManager

# Create agents
coder = AssistantAgent(name="Coder", ...)
reviewer = AssistantAgent(name="Reviewer", ...)
tester = AssistantAgent(name="Tester", ...)

# Create group chat
group_chat = GroupChat(
    agents=[coder, reviewer, tester],
    messages=[],
    max_round=20
)

# Manager coordinates who speaks next
manager = GroupChatManager(
    groupchat=group_chat,
    llm_config=llm_config
)

coder.initiate_chat(manager, message="Build API")
```

46/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

How is code execution handled in Autogen?

```
user_proxy = UserProxyAgent(  
    name="User",  
    code_execution_config={  
        "work_dir": "output",  
        "use_docker": True, # RECOMMENDED!  
        "timeout": 60,  
        "last_n_messages": 3  
    }  
)
```

PYTHON

WORKFLOW

1. Assistant generates code → 2. Proxy extracts & executes → 3. Output returns → 4. Iterate if errors

⚠ **Always use Docker** isolation to prevent malicious code from affecting host system!



What is human-in-the-loop in Autogen?

PYTHON

```
# ALWAYS - human approves every message
user_proxy = UserProxyAgent(
    name="Human",
    human_input_mode="ALWAYS",
    max_consecutive_auto_reply=0
)

# TERMINATE - auto until termination
user_proxy = UserProxyAgent(
    name="Human",
    human_input_mode="TERMINATE",
    is_termination_msg=lambda msg:
        "COMPLETE" in msg["content"]
)

# NEVER - fully autonomous
human_input_mode="NEVER"
```

USE CASES

48/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

How are termination conditions defined in Autogen?

PYTHON

```
# Termination message check
def is_termination(msg):
    content = msg.get("content", "")
    return "TASK_COMPLETE" in content

agent = AssistantAgent(
    name="Assistant",
    is_termination_msg=is_termination
)

# Max consecutive auto replies
user_proxy = UserProxyAgent(
    max_consecutive_auto_reply=10
)

# Max rounds in group chat
group_chat = GroupChat(agents=[...], max_round=15)
```

49/50



Naved Khan
Gen AI Engineer

[+ Follow](#)

What are best practices for Autogen?

- ✓ **Termination Conditions:** Always set multiple limits
- ✓ **Docker for Code:** Never execute without sandboxing
- ✓ **Clear Roles:** Distinct responsibilities in group chats
- ✓ **Human Oversight:** Checkpoints for sensitive actions
- ✓ **Monitor Tokens:** Long conversations = high costs
- ✓ **Error Iteration:** Let agents discuss & fix errors
- ✓ **Use Caching:** Reduce API costs in development

50/50



Naved Khan
Gen AI Engineer

+ Follow



THAT'S ALL!

Thank You for Reading!

Save this for your interview prep. Share
with someone who needs it!



Save



Repost



Comment



Naved Khan
Gen AI Engineer

+ Follow